

Lista de Exercícios I

1. Explique a diferença entre código síncrono e código assíncrono em JavaScript e também responda:
 - a) Por que o modelo síncrono pode ser um problema em aplicações backend?
 - b) Cite dois exemplos de operações que normalmente são executadas de forma assíncrona.
2. Explique o que é uma Promise em JavaScript e também responda:
 - a) Qual problema as Promises resolvem.
 - b) Quais são os três estados possíveis de uma Promise.
 - c) Como uma Promise pode ser consumida utilizando then() e catch().
3. Sobre async/await e tratamento de erros:
 - a) O que faz a palavra-chave async em uma função JavaScript?
 - b) Qual é a função da palavra-chave await?
 - c) Por que é importante utilizar try/catch em operações assíncronas?
 - d) Cite dois exemplos de situações em backend onde operações assíncronas são comuns.

Exercício Prático

4. Mini Backend em JavaScript:
Crie um pequeno sistema que utilize Promise, async/await, try/catch, modularização e import/export.
 - a) Crie uma função chamada buscarMensagem() que retorne uma Promise que resolve após alguns segundos com a mensagem 'Operação concluída'.
 - b) Consuma essa Promise utilizando then() e catch(), exibindo o resultado no console.
 - c) Reescreva o código anterior utilizando async/await.
 - d) Crie uma função chamada buscarUsuario() que retorne uma Promise que gere erro. Utilize async/await e try/catch para capturar o erro e mostrar a mensagem no console.
 - e) Agora simule a estrutura de um backend utilizando os arquivos:
userRepository.js
userService.js
userController.js
app.js

f) No arquivo `userRepository.js`, crie um array de usuários e uma função `buscarUsuario(id)` que retorne o usuário correspondente.

g) No arquivo `userService.js`, importe o repository e crie a função `obterUsuario(id)`.

h) No arquivo `UserController.js`, importe o service e crie a função `buscarUserController(id)` que mostre o resultado no console.

i) No arquivo `app.js`, importe o controller e execute `buscarUserController(1)`.